CSCE 735 PARALLEL COMPUTING - Major Project

Yashas Bharadwaj
UIN: 635009332

Objective:

The project focused on developing a parallel GPU implementation of the Cholesky factorization algorithm.

The core of the parallel GPU code lies in the parallel_cholesky_factorization() function. This function executes the Cholesky factorization iteratively, processing one column (pivot) of the matrix in each iteration. For every pivot column, the function sequentially calls three distinct Kernel Functions:

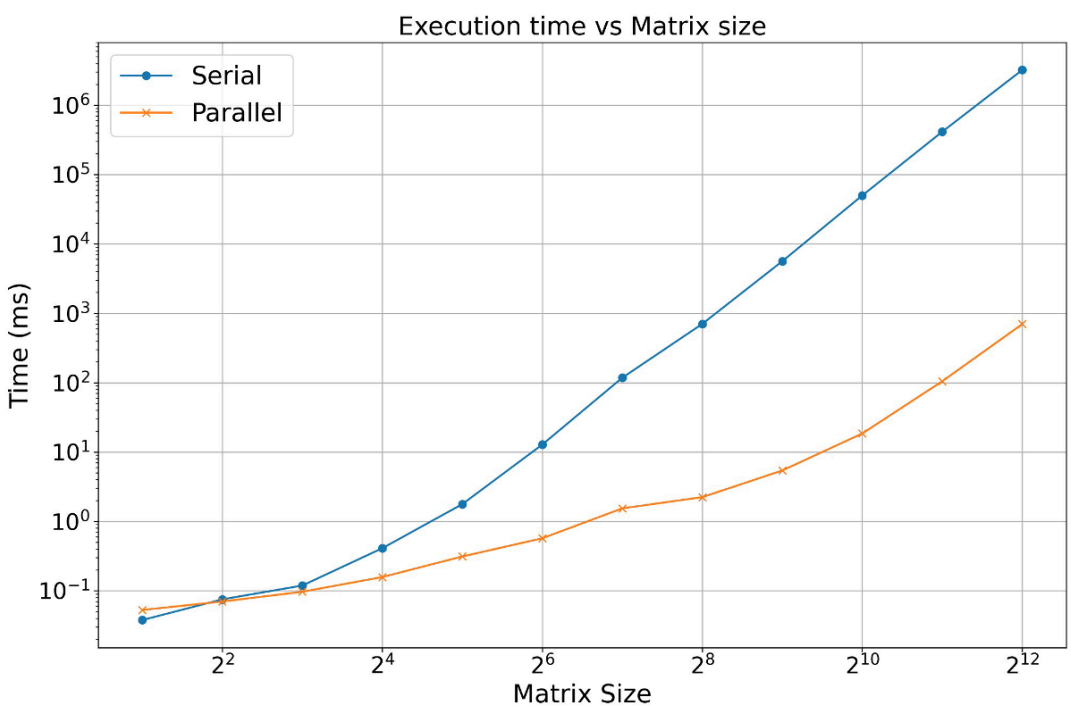
1. **device_cholesky_normalize**: Calculates the square root of the diagonal element and normalizes the elements in the corresponding row by dividing them by this value.
2. **device_cholesky_submat_update**: Updates the remaining submatrix by subtracting the outer product of the current pivot row and column.
3. **device_cholesky_lower_traingale_update**: Enforces the lower triangular structure by setting elements below the diagonal to zero.

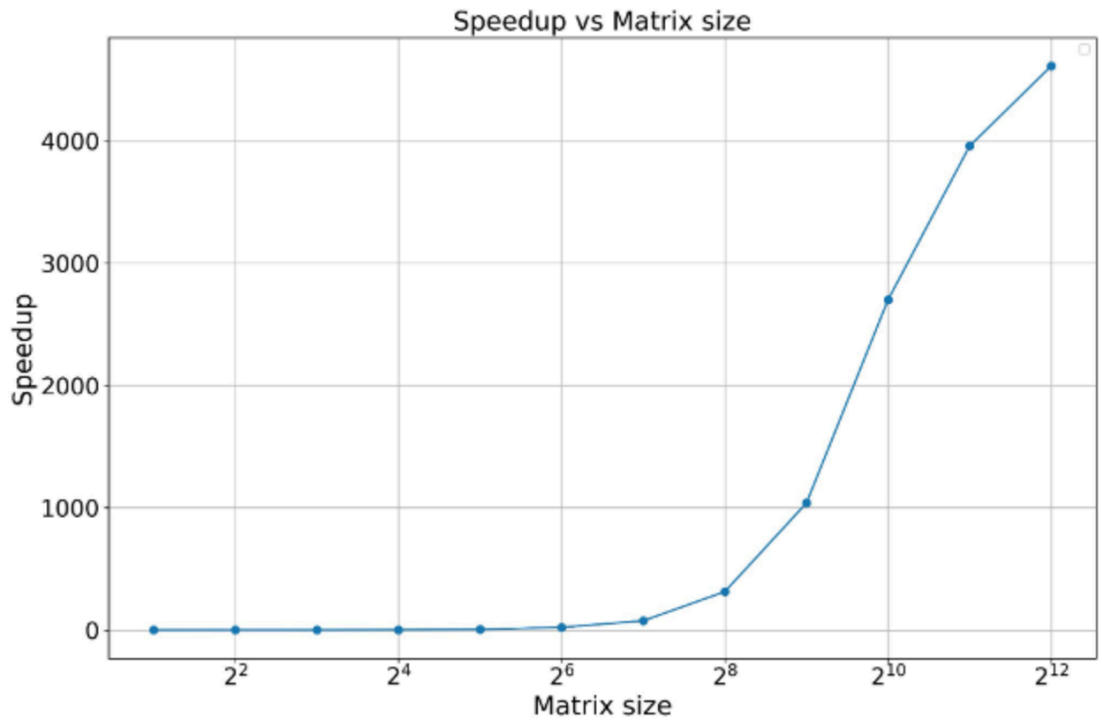
A key optimization was implemented through the dynamic calculation of grid dimensions based on the matrix and block sizes, ensuring efficient thread utilization on the GPU. Specifically, 1D grid structures were employed for the normalization and lower triangle update operations, as these operate on single rows. In contrast, a 2D grid structure was used for the submatrix update, which involves a two-dimensional area.

1. Effect of Varying Matrix Size:

The matrix size was varied across a range from 2 to 4096. During this variation, the following parameters were kept constant: the block size in the 1D grid was 1024, and the dimensions of the 2D grid were 32*32 (i.e., x dimension of 32 and a y dimension of 32, totaling 1024).

Matrix Size	Serial Execution Time (ms)	Parallel Execution Time (ms)	SpeedUp
2	0.0379	0.0529	0.7159090908
4	0.0751	0.0698	1.075822605
8	0.1189	0.0968	1.228541883
16	0.4104	0.1574	2.605079364
32	1.7715	0.3119	5.68152662
64	12.805	0.5706	22.43953733
128	117.7070	1.5401	76.42821895
256	702.9994	2.2347	314.5832998
512	5625.7738	5.4132	1039.269545
1024	49930.2278	18.4851	2701.106653
2048	414389.126	104.6818	3958.559415
4096	3220160.30	698.0915	4612.805413





In the above graphs, both the x and y axis are in logarithmic scale. And below are the observations for the same:

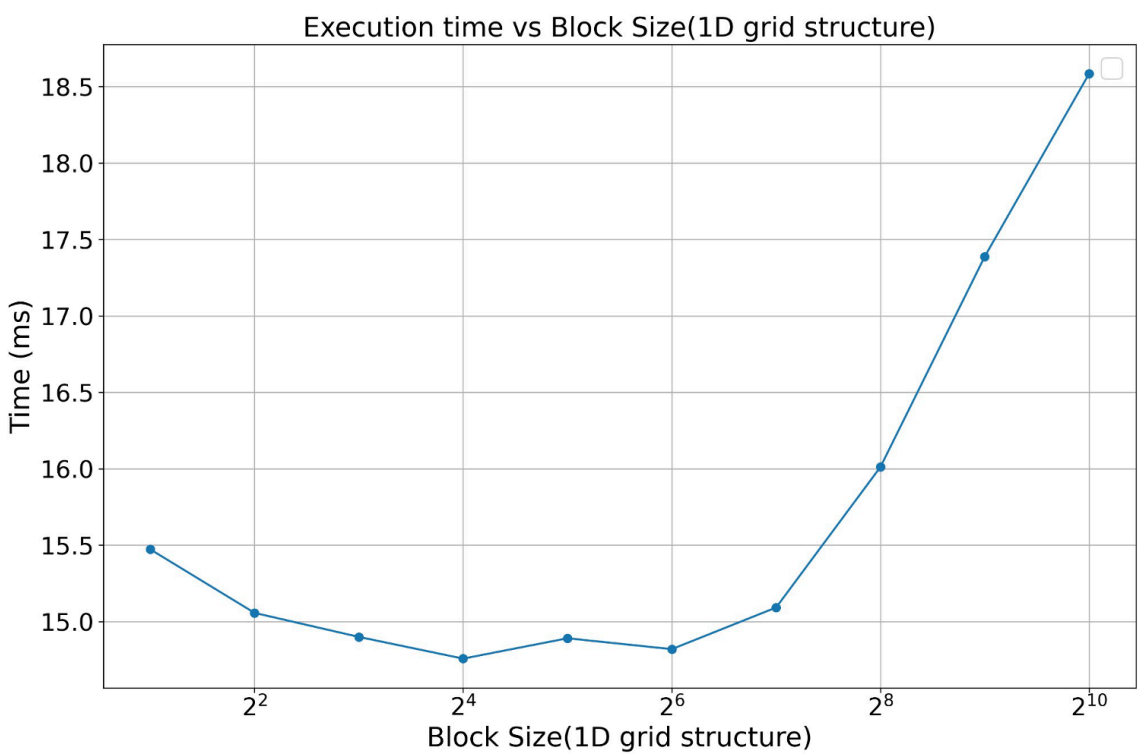
- The parallel execution time for smaller matrices is greater than the serial implementation due to the overhead associated with launching kernel functions and memory transfers.
- However, the benefits of parallel implementation become significant starting with a matrix size of 64, achieving a speedup of 22 times. The power of GPU computation becomes evident beyond a size of 128, where speedups exceed 75x. As the matrix size continues to increase, the GPU demonstrates a drastic increase in speedup, reaching over 4500x for a 4096 x 4096 matrix.
- This superior performance for larger matrices is attributed to more regular and predictable memory access patterns, which allow for efficient reuse of elements within blocks by leveraging the cache and optimizing shared memory utilization.
- Therefore, we conclude that the parallel GPU code is highly effective and recommended for obtaining superior performance when dealing with larger matrices.

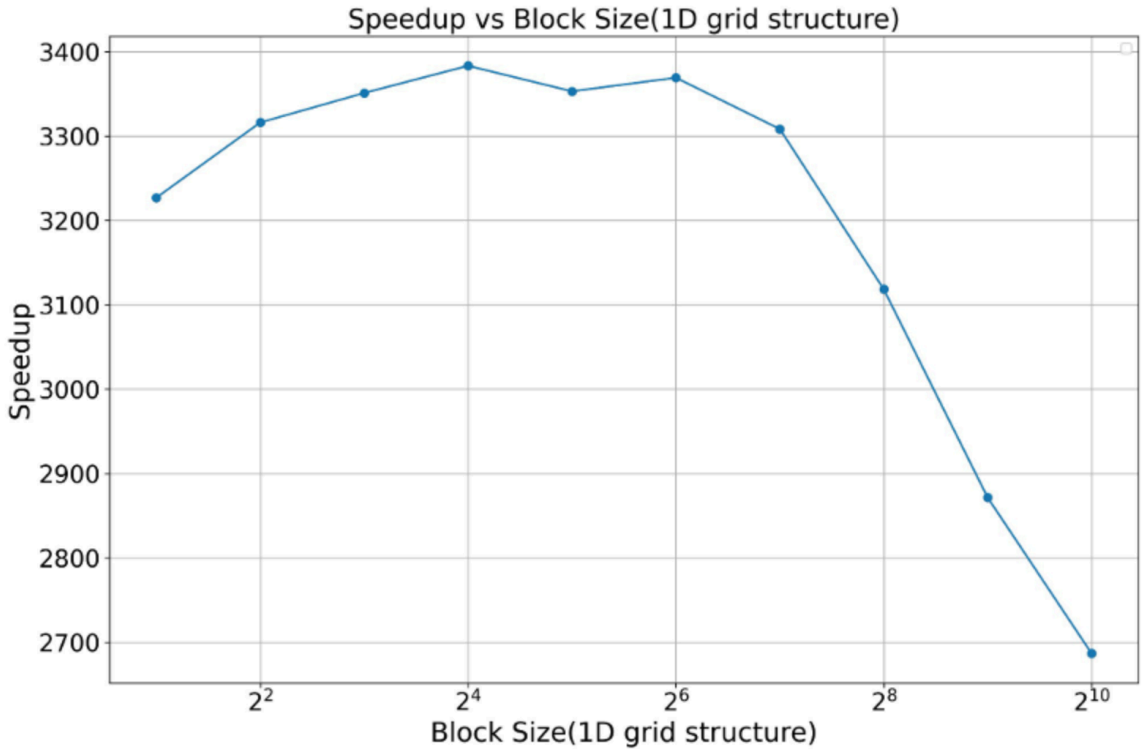
2. Effect of Block Size in 1D Grid:

The following parameters were used for the experiment:

- Matrix Size: 1024
- 1D Grid Block Size: Varied from 2 to 1024
- 2D Grid Dimensions: 32 (x-dimension) by 32 (y-dimension), totaling 1024
- Serial Time Baseline for Speedup Calculation: 49930.2278 ms (for a 1024 matrix size)

Block Size in 1D	Parallel Execution Time (ms)	SpeedUp
2	15.4722	3227.072031
4	15.0565	3316.212812
8	14.8997	3351.179354
16	14.7572	3383.448525
32	14.8901	3353.182352
64	14.8198	3369.270246
128	15.0922	3308.456077
256	16.012	3118.495192
512	17.389	2871.533620
1024	18.5837	2686.789788





Observations:

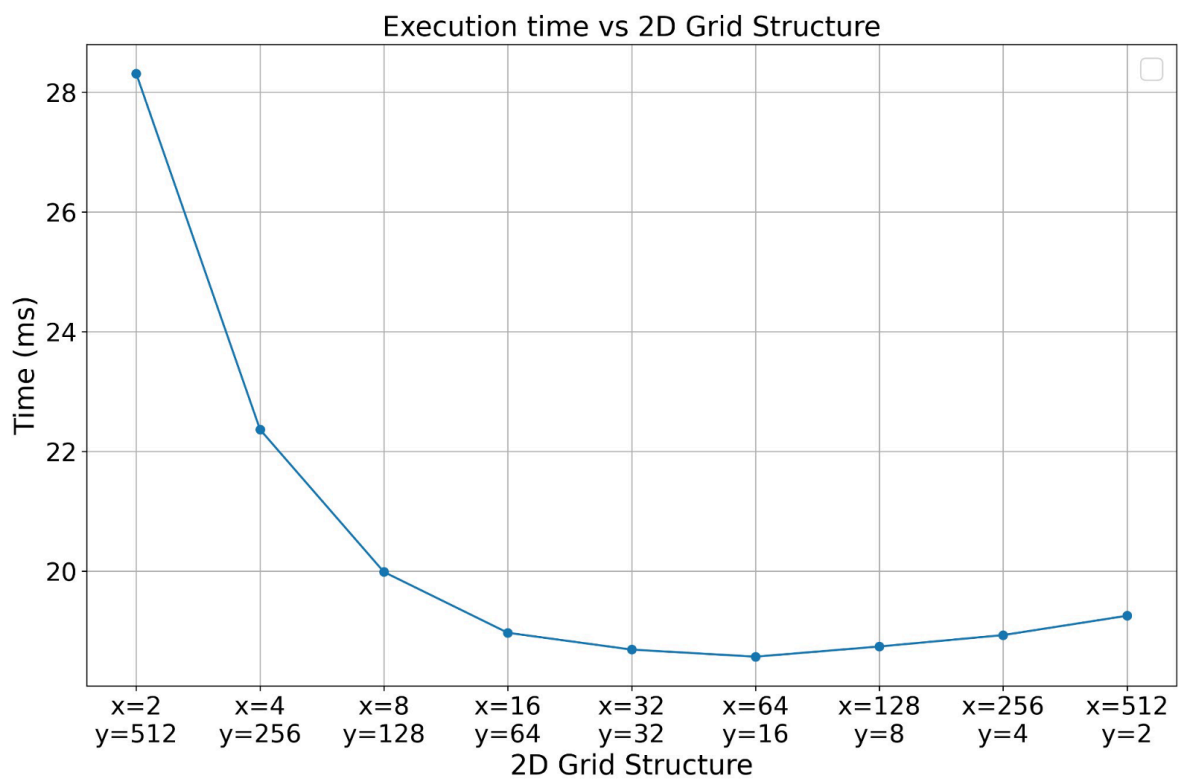
Analysis of the results indicates a clear relationship between block size and runtime. Initially, runtime decreases as the block size increases, suggesting a benefit from increased parallelism. The optimal performance was achieved at a block size of 16, yielding a significant speedup of 3383. This optimal point demonstrates the best balance between parallel processing gains and the associated overhead.

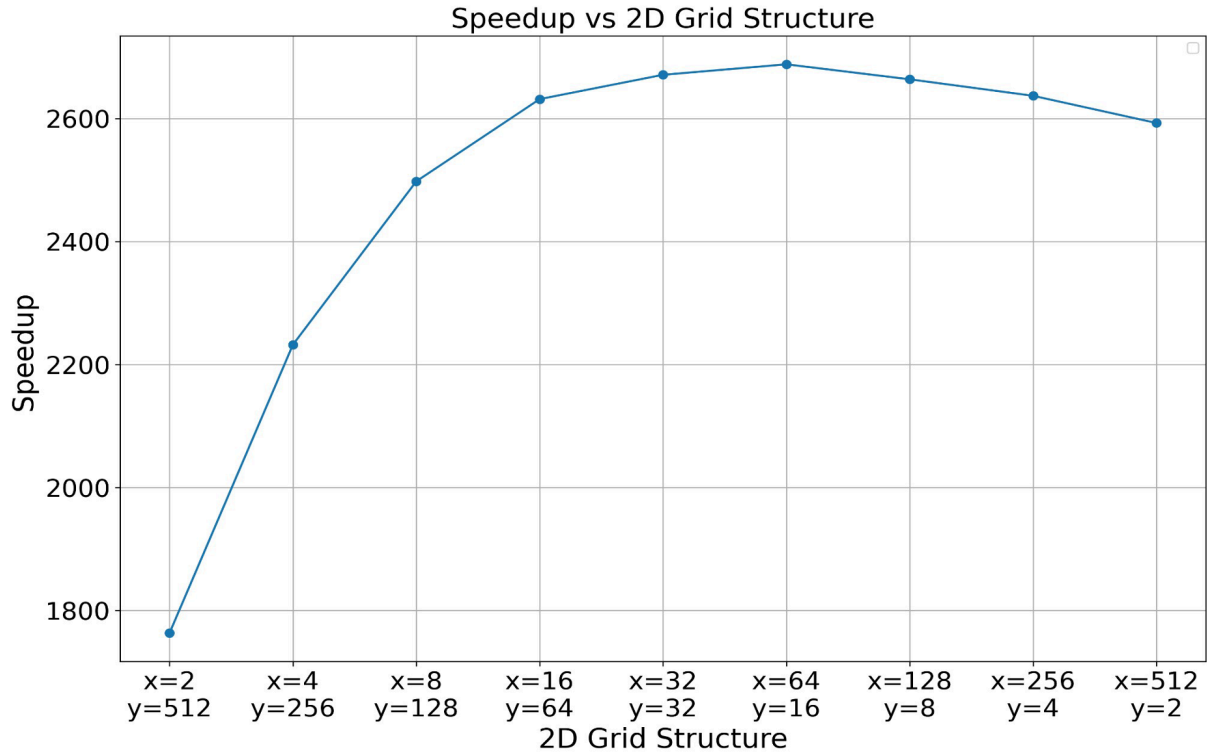
However, performance begins to degrade when the block size exceeds 128. This degradation is likely due to inefficient work distribution across blocks or lower occupancy of the Streaming Multiprocessors (SMs). Therefore, while the impact of block size in a 1D grid may not be uniformly dramatic, selecting an appropriate block size is crucial to ensure that the gains from parallelism effectively outweigh the inherent overhead.

3. Effect of Dimensions in 2D Grid:

The project involved varying the dimensions (x and y) of a 2D grid such that their product remained constant at 1024. This was performed while keeping the matrix size at 1024 and maintaining a 1D block size of 1024. The serial execution time of 49930.2278 ms, measured for the 1024 matrix size, was used as the baseline for calculating speedup.

x Dimension in 2D Grid	y Dimension in 2D Grid	Parallel Execution Time (ms)	SpeedUp
2	512	28.3102	1763.477722
4	256	22.3998	2232.346595
8	128	19.8992	2497.985146
16	64	18.9562	2631.687981
32	32	18.6212	2671.251222
64	16	18.9722	2688.294249
128	8	18.7568	2663.897314
256	4	18.9332	2637.094861
512	2	19.2569	2592.835112





Observations:

- **Impact of Thread Distribution:** The distribution of threads across dimensions significantly affects performance. Experiments with 2D grid structures indicate that a more balanced distribution, close to a square dimension (16x64, 32x32, 64x16), provides the best results.
- **Optimal Thread Block Shape:** Surprisingly, the 64x16 dimension outperformed the square 32x32 dimension, suggesting that rectangular shapes can sometimes offer better memory access patterns than square ones.
- **Memory Access Efficiency:** Row-oriented memory access patterns were found to be more efficient than column-oriented ones.
- **Worst-Case Scenario:** Highly skewed matrices, such as 2x512, exhibit the worst performance due to non-coalesced access patterns, leading to inefficient memory transactions.

Code Compilation Instructions:

Follow these steps to successfully compile and run my GPU-accelerated parallel Cholesky factorization algorithm on the Grace cluster:

- **Load Module:**
  - Command: module load intel/2023a CUDA/12.2
- **Compile:**
  - Compile cholesky_gpu_parallel.cu.
  - Command: nvcc -o cholesky_gpu_parallel.exe cholesky_gpu_parallel.cu
- **Run Command:**
  - Format: ./cholesky_gpu_parallel.exe <matrix size> <block 1d grid> <block 2d grid x> <block 2d grid y>
  - **Arguments and Constraints:**
    i. <matrix size>: Square matrix dimension (Max 4096).
    ii. <block 1d grid>: 1D grid block size (1 to 1024).
    iii. <block 2d grid x>: 2D grid block x-dimension (x*y <= 1024).
    iv. <block 2d grid y>: 2D grid block y-dimension.
- **Example:**
  - Command: ./cholesky_gpu_parallel.exe 1024 1024 32 32

Note: If a CUDA memory error occurs on the Grace login shell, retry the command or use an sbatch job script.